

LLM.

→ large language Model.

→ Transformer architecture.

→ Self-Attention Mechanism

Attention is
All you
need.

Bank. → I went to Bank yesterday to deposit
some money.

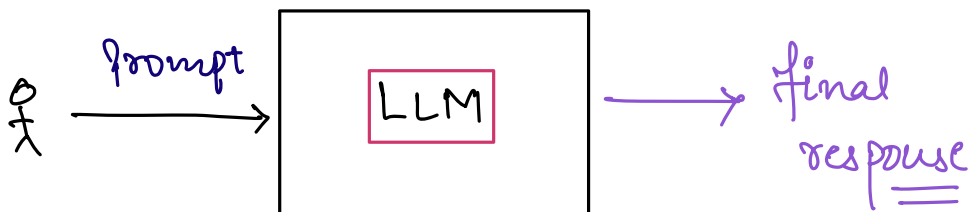
Bank. → I sat on river Bank for some time.

LLM

→ Trained on huge volumes of internet data.
till some cut-off date.

Generative AI

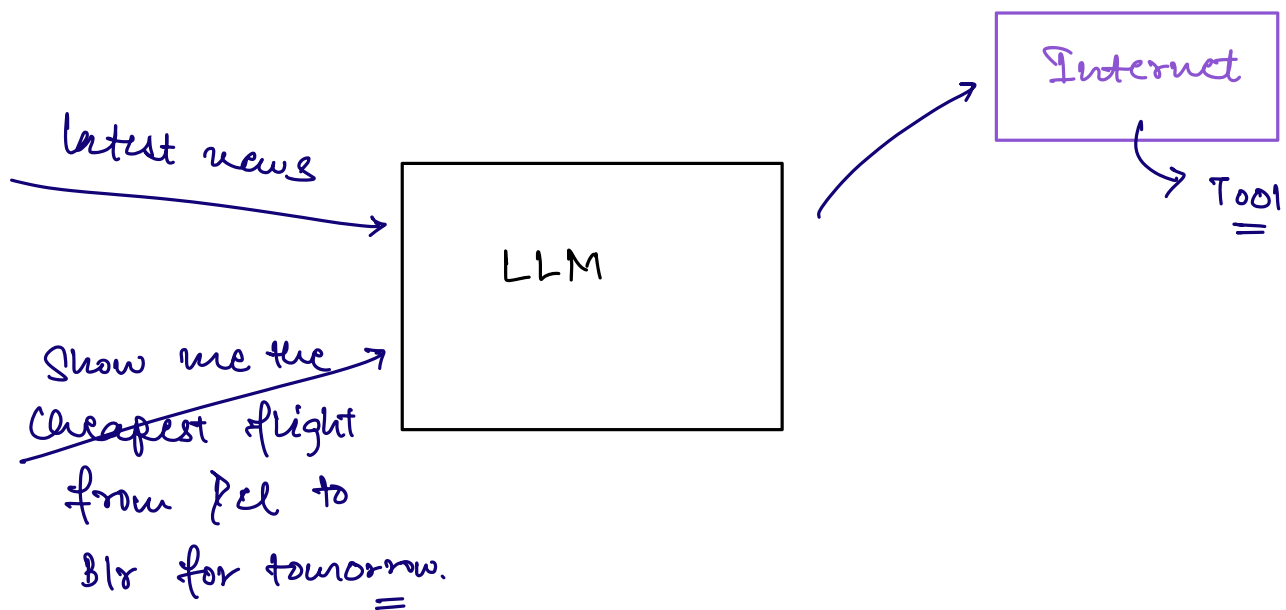
→ Type of AI where LLM generates the response
based on its pre-trained information.



Prompt → Model → Output.

AI Agent

↳ Can take decisions & execute the decisions as well.



Agentic AI

Debugging with AI.

"When you paste a stack trace or a broken function into an AI tool and ask 'why is this failing', the model is doing something specific: pattern-matching your code and error message against the enormous amount of code, bugs, and fixes it saw during training. It is not executing your code. It is not running a debugger. It is predicting, based on patterns, what is most likely wrong.

This matters because it shapes what AI debugging is good at and what it is not. It is very good at common, well-documented bugs — off-by-one errors, null reference issues, common misuse of a popular library, syntax errors, obvious logic mistakes. It is much weaker at bugs that depend on your specific runtime state, your specific data, or an interaction between multiple parts of your system that only manifests in production."

Structuring a Debugging Prompt.

"This connects directly to prompt engineering. A debugging request is a prompt, and it benefits from the same structure — context, instruction, and constraints. The context is your error message and relevant code. The instruction is what you actually want: identify the bug, explain why it happens, suggest a fix. The constraint might be: do not rewrite the whole function, just point out the issue."

WEAK:

"My code doesn't work, here's the error: `TypeError: unsupported operand type(s) for +: 'int' and 'str'`"

Response will be generic – an explanation of what this error type usually means, without being able to point to the actual cause.

STRONG:

"This function raises `'TypeError: unsupported operand type(s) for +: 'int' and 'str''` when called with `calculate_total([10, 20, '30'])`.

```
def calculate_total(prices):  
    total = 0  
    for price in prices:  
        total += price  
    return total
```

What is causing this, and what is the safest fix?"

Now the model has the actual code, the actual error, and the actual input that triggers it. It can identify precisely that the list contains a string where a number was expected, and suggest either validating input types or converting the value.

- AI debugging works by pattern-matching your code and error against training data, not by executing your code
- It excels at common, well-documented bugs and struggles with bugs dependent on specific runtime state or complex system interactions
- A strong debugging prompt includes the exact error, the relevant code, and the specific input that triggers it — the same discipline as debugging without AI
- Treat every AI debugging suggestion as a hypothesis to test, not a verdict to trust — confident-sounding explanations can still be wrong

Code Review with AI

Code review dimensions:

1. Correctness — does the code do what it claims to do?
2. Readability — can someone else understand this in six months?
3. Consistency — does it follow the codebase's existing patterns?
4. Risk — security, performance, edge cases (Block 4)
5. Maintainability — will this be easy to change later? (Block 5)

- Code review evaluates correctness, readability, consistency, risk, and maintainability — not just whether the code runs
- Correctness is usually the easiest dimension to verify; readability and consistency matter for long-term codebase health
- Naming, idiom usage, and consistency with existing patterns are areas where AI review tools are consistently strong, since these patterns are heavily represented in training data

Risk categories to review for:

- Security - injection, exposed secrets, missing authorization checks
- Edge cases - empty inputs, null values, boundary conditions
- Error handling - unhandled exceptions, silent failures
- Performance - N+1 queries, unbounded loops, missing pagination
- Concurrency - race conditions, unsafe shared state

WEAK PROMPT:

"Review this code."

STRONG PROMPT:

"Review this FastAPI route specifically for security risks (authorization, injection, data exposure) and performance risks (unbounded queries, missing pagination). List each risk found, why it matters, and a suggested fix."

- Refactoring changes code structure without changing external behavior; this is distinct from bug fixes and feature additions
- AI tools are strong at pattern-based refactors — extracting functions, removing duplication, flattening nested conditionals
- "Same behavior" is a claim, not a guarantee — verify a refactor with existing tests, or write tests first if none exist
- Good test coverage is what makes accepting AI refactoring suggestions low-risk; without it, refactoring is inherently riskier regardless of who or what suggested it

Lecture Summary.

Debugging with AI

- AI debugging pattern-matches your code and error against training data; it does not execute your code
- Strong for common, well-documented bugs; weak for bugs dependent on specific runtime state or complex system interactions
- A strong debugging prompt includes the exact error, relevant code, and the specific triggering input
- Every AI debugging suggestion is a hypothesis to test, not a verdict to trust

Code Review Fundamentals

- Code review covers correctness, readability, consistency, risk, and maintainability
- Correctness is usually the easiest dimension to verify; readability and consistency matter for long-term codebase health
- Naming and idiom consistency are areas where AI review is consistently strong

Identifying Risks

- Risk categories include security, edge cases, error handling, performance, and concurrency — missed entirely by a correctness-only review
- Many risks connect directly to earlier course material — authorization, N+1 queries, injection
- Prompting for specific, named risk categories with a structured response format produces far more useful review results than a generic request

Refactoring Suggestions

- Refactoring changes structure without changing behavior; distinct from bug fixes and feature work
- AI tools are strong at pattern-based refactors, since these patterns are heavily represented in training data
- "Same behavior" must be verified with tests, existing or newly written, before accepting a refactor

AI-Assisted Code Review in Practice

- The most effective workflow uses AI as a first-pass filter before human review, not as a replacement
- This frees human reviewer attention for architecture, priorities, and judgment calls AI cannot make
- AI review lacks team context and system-wide understanding, and can produce confident but incorrect comments